Plan Offero — Parte 3 de 10

Base técnica, entornos y acuerdos de trabajo · Versión 2 · 30 de octubre de 2026 · Entrega del MVP: viernes 29 de enero de 2027 [SUPUESTO]

> Sin código: aquí van tablas, archivos, comandos y reglas. Lo que ya está en las guías individuales (Anexos B a G) se resume y se referencia para que exista una sola fuente.

6. Base técnica y entornos

Estado al 29 oct: repositorio protegido, colaboradores invitados, etiqueta `prototipo-v0`, y dos proyectos de Supabase (`offero-dev` y `offero-prod`) con RLS automático.

6.1 Orden de creación de las tablas

El orden sigue las dependencias: ninguna tabla se crea antes que las tablas a las que apunta. Todas se crean primero en `offero-dev`; Wilson (responsable de la base de datos, D-10) las revisa y las aplica en `offero-prod` antes de fusionar el PR.

#	Tabla u objeto	Depende de	Responsable	En dev	Sprint
1	`profiles` + trigger de registro + funciones `is_admin()` y `get_my_role()` + vista `public_profiles`	auth.users (Supabase)	Diego	mar 3 nov	0
2	`categories` + carga de categorías reales	—	Daniel	mar 10 nov	1
3	`requests`	profiles, categories	Wilson	mié 11 nov	1
4	`businesses`	profiles, categories	Daniel	jue 12 nov	1
5	`reports`	profiles	Iker	dom 15 nov	1
6	`quotes`	requests, businesses	Diego	mar 17 nov	2
7	`messages` + función `can_message`	profiles, requests, quotes	Wilson	dom 13 dic	5
8	`reviews` + vista de promedio (`business_ratings`)	profiles, businesses, requests, quotes	Iker	dom 13 dic	5
9	Función de aceptar cotización	quotes, requests	Diego	vie 4 dic	4
10	Políticas de admin en tablas ajenas (ocultar)	todas las anteriores	Iker	dom 13 dic	5
11	`profiles.is_suspended` y política de cambio de rol	profiles	Iker con Diego	8 – 15 ene	7

Diseño en conjunto antes de crear: el jueves 5 de noviembre (2 h, los 4) se repasan todas las tablas, sus relaciones y los valores de estado acordados (Anexo D de las guías). Cambiar una tabla después de que otros la usan cuesta mucho más.

6.2 Cuándo se configuran las políticas RLS

- **Siempre en el mismo PR que crea la tabla.** El RLS automático la activa al crearla; sin políticas, la tabla responde vacío para todos. Una tabla sin sus políticas de SELECT, INSERT, UPDATE y DELETE (aunque sea «nadie») no se fusiona.
- **Políticas que llegan después**, porque dependen de tablas futuras:

Política	Tabla	Cuándo	Quién
Los proveedores siguen viendo las solicitudes que cotizaron	`requests`	Cuando exista quotes (sáb 28 nov)	Wilson con Diego
El admin crea, renombra y borra categorías	`categories`	jue 19 nov	Iker o Daniel (acordarlo)
El admin oculta negocios, solicitudes y reseñas	businesses, requests, reviews	dom 13 dic	Iker; aprueba el dueño de cada tabla
El admin cambia roles; cuentas suspendidas sin acceso	`profiles`	8 – 15 ene	Iker con Diego

- **Revisión:** Wilson revisa el esquema (tipos, claves, restricciones) e Iker las políticas con la checklist; los dos aprueban el PR. Las **vistas no tienen RLS**: Wilson e Iker revisan cada vista antes de fusionar. En el Sprint 0 y el Sprint 1 Diego revisa junto con Wilson.
- **Prueba mínima de cada política:** dos cuentas de roles distintos en dev; la cuenta «ajena» intenta leer, crear, editar y borrar.

6.3 Entornos y flujo de trabajo

Entorno	Dónde corre	Base de datos	Quién lo usa	Para qué
Desarrollo local	Tu computador (`npm run dev`, localhost:3000)	`offero-dev`	Cada integrante	Programar y probar mientras construyes
Preview (vista previa)	Vercel: un enlace por cada PR	`offero-dev`	Autor y revisor	Probar el cambio en un servidor real antes de fusionar
Producción	Vercel: URL principal	`offero-prod`	Empresas del piloto, profesor	Solo lo que ya está en main y probado

Recorrido de un cambio:

1. Creas tu rama y programas en local contra `offero-dev`.
2. Si tu cambio incluye SQL, lo aplicas en dev desde tu archivo de `supabase/sql/` y avisas en Discord.
3. Abres el PR: Vercel genera el Preview automáticamente.
4. El revisor prueba en el Preview (camino feliz + permisos con otra cuenta) y aprueba.
5. Si hay SQL, Wilson (o Diego, de respaldo) lo aplica en `offero-prod` **antes** de fusionar; luego Diego fusiona con merge commit, para que el código nuevo no llegue a una base sin sus tablas.
6. Vercel publica en producción. El autor hace una verificación de humo: entra con su cuenta real, navega, y revisa la consola.

> **Ojo:** local y Preview comparten `offero-dev`. Un cambio de esquema en dev afecta a los cuatro al instante. Por eso en dev solo se hacen cambios **que agregan** (tablas, columnas, políticas nuevas). Borrar o renombrar algo que otro usa se acuerda antes en Discord.

6.4 Variables de entorno y claves

Variable o clave	Dónde va	¿Se sube a GitHub?	Quién la tiene
URL del proyecto (`NEXT_PUBLIC_SUPABASE_URL`**)**	`.env.local` (dev) · Vercel Preview (dev) · Vercel Production (prod)	No (solo el nombre, en `.env.example`)	Todos: la de dev. La de prod solo está en Vercel

Variable o clave	Dónde va	¿Se sube a GitHub?	Quién la tiene
Clave pública (anon o publishable)	Igual que la URL	No (solo el nombre)	Todos: la de dev. La de prod solo está en Vercel
Clave secreta (`service_role` o secret)	**En ningún lado de la app** [SUPUESTO: el MVP no la necesita; todo pasa por RLS]	**Nunca**	Solo Diego, en su gestor de contraseñas
Contraseñas de las bases de datos	En ningún lado de la app	**Nunca**	Diego y Wilson (la de prod, por mensaje privado)
Acceso al panel de Vercel	—	—	Solo Diego (plan Hobby)

- El nombre exacto de la variable de la clave pública lo fija Diego en `.env.example` según la clave que muestre el panel (Supabase usa hoy «publishable» y antes «anon»). Nadie inventa otro nombre.
- En Vercel, el entorno «Development» no se usa; solo Preview y Production.
- Después de cambiar una variable en Vercel hay que volver a desplegar; después de editar `.env.local`, reiniciar `npm run dev`.
- Todo lo que empieza por `NEXT_PUBLIC_` es visible en el navegador: ahí solo van la URL y la clave pública.
- Las claves de dev se pasan por mensaje privado, nunca en el canal general ni en un PR. Si una clave secreta se filtra, Diego la regenera en el panel ese mismo día.

6.5 Compartir Supabase entre 4 personas

- **Un solo proyecto de pruebas (`offero-dev`) para los cuatro.** No hace falta un tercer proyecto: dev ya es el proyecto de pruebas.
- **Todos entran a la organización con rol Developer.** En el plan gratuito ven también `offero-prod`: por acuerdo, **solo Wilson y Diego ejecutan SQL en prod** (D-10).
- **Todo cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql`. Nadie cambia tablas desde el editor visual sin dejar su archivo; así prod queda idéntico a dev.
- **Nunca se edita un archivo ya aplicado:** si hay que corregir, se crea uno nuevo.
- **Avisar antes de aplicar en dev:** «voy a crear la tabla X» en #prs. Si dos personas tocan la misma tabla, se coordinan antes.
- **Cuentas de prueba con alias de correo:** por ejemplo `tunombre+cliente1@gmail.com` y `tunombre+proveedor1@gmail.com` (Gmail entrega en tu bandeja). Así cada uno tiene varias cuentas de distintos roles.
- **Datos de prueba marcados:** títulos que empiezan por «[PRUEBA]» para saber qué se puede borrar.
- **Proyectos pausados:** [Verificar] en el plan gratuito los proyectos pueden pausarse por inactividad. Diego revisa `offero-prod` antes de cada demo y al volver del descanso de Navidad.

6.6 Probar sin romper la regla de «sin datos ficticios»

Qué	Dónde	Regla
Probar formularios, permisos y casos de error	Local y Preview (`offero-dev`)	Datos inventados permitidos, marcados con [PRUEBA]
Prueba cruzada al cierre del sprint	Preview o local (dev)	Igual; quien prueba no es quien construyó

Qué	Dónde	Regla
Verificación después de publicar	Producción	Solo de lectura: entrar con tu cuenta real y navegar. No crear registros
Demostración a las empresas y al profesor	Producción	Solo datos reales: cuentas, negocios, solicitudes y cotizaciones de las empresas del piloto
Categorías iniciales	Dev y prod	Reales en ambos (las define el PO con las entrevistas)
Si por error se crea un dato inventado en prod	Producción	Wilson lo borra ese día y se anota en la bóveda

Un módulo solo se marca «Terminado» cuando su operación principal funciona en producción **con datos reales** (por ejemplo, M4 con una cotización real de una empresa del piloto). Por eso el PO debe tener las empresas listas antes del Sprint 3.

7. Acuerdos de trabajo (Working Agreements)

Son las reglas que el equipo se compromete a cumplir. Se aceptan en la primera Sprint Planning (dom 8 nov) y se revisan en cada Retrospectiva.

7.1 Definition of Ready (DoR)

La DoR es la lista de condiciones para que una historia pueda entrar a un sprint. Si no las cumple, se refina antes.

- Está escrita como «Como [rol], quiero [acción], para [beneficio]».
- Tiene criterios de aceptación Dado / Cuando / Entonces, incluidos los de permisos.
- Está estimada (1, 2, 3, 5 u 8 puntos); si pasa de 8, se divide.
- Sus dependencias están terminadas o tienen fecha dentro del mismo sprint (por ejemplo, la tabla de la que depende).
- Tiene responsable con nombre y el responsable entiende qué hay que hacer.
- Si toca la base de datos, se sabe qué tabla, qué campos y qué políticas (o quién las define).
- No hay una decisión de producto pendiente que la bloquee (sección 11).

7.2 Definition of Done (DoD) en tres niveles

Nivel	Está hecho cuando…
Historia	Cumple sus criterios de aceptación · PR revisado por al menos un compañero y fusionado · funciona en el Preview y luego en producción · permisos probados con dos cuentas de roles distintos · validación en el servidor · sin errores en la consola · SQL en un archivo y aplicado en dev y prod · documentación actualizada si cambió algo
Sprint	Todas las historias terminadas cumplen su DoD · la prueba cruzada del sprint está hecha · lo terminado se mostró en la Review · los errores encontrados están registrados · el documento de contexto refleja el estado (secciones 8, 9 y 10) · la velocidad quedó anotada
Módulo (regla del documento)	Funciona en la versión publicada en Vercel · sus operaciones principales están probadas por otra persona · no depende de datos ficticios en producción · la sección 8 dice Terminado y Publicado: Sí

7.3 Flujo de Git y GitHub

El paso a paso con comandos está en el Anexo C de cada guía. Aquí van las reglas:

Tema	Acuerdo
Rama main	Protegida: solo cambia por PR con 1 aprobación; sin force push ni borrado; fusión solo con merge commit (la hace Diego)
Ramas	Una por tarea, desde main actualizada. Nombre: `tipo/mN-descripcion` con tipo feat, fix, chore o docs. Ejemplo: `feat/m3-formulario-solicitud`
Commits	Pequeños y en español: `tipo(módulo): qué hace`. Ejemplo: `fix(m2): corrige filtro por categoría vacío`
Pull requests	Menos de un día de trabajo; con la plantilla (qué cambia, cómo probarlo, capturas, checklist); enlazan la tarea con «Cierra #N»
Revisión	Al menos un compañero. Si trae SQL, revisan Wilson (esquema) e Iker (RLS); si toca autenticación o archivos compartidos, revisa Diego. Respuesta en menos de 24 h. Se corrige con nuevos commits en la misma rama
Actualizar la rama	Cada día: traer main y fusionarla en tu rama. Cuanto más vieja la rama, más conflictos
Conflictos	Resolverlos en VS Code eligiendo el cambio correcto; si no entiendes el cambio del otro, se lo preguntas a su autor. Conflicto en `package-lock.json`: aceptar la versión de main y volver a ejecutar `npm install`
Prohibido	Commits directos a main · `git push --force` · subir `.env.local`, `node_modules` o claves · archivos .rar o .zip con código

7.4 Cómo no pisarse en el mismo repositorio

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisar en Discord antes de cambiarlos; cambios pequeños; Daniel revisa
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/` (esquema)	Dueño de cada tabla; aprueba Wilson	No editar archivos ya aplicados; avisar antes de aplicar en dev
`package.json` y `package-lock.json`	Todos, con acuerdo	No instalar paquetes sin acordarlo (restricción de stack)
`src/components/ui/` (componentes compartidos)	Quien crea el componente	Si cambias ReportButton o CategoryFilter, avisa a quienes los usan
`docs/` (bóveda de Obsidian)	Daniel coordina; cada uno su módulo	Actualizar tu nota de módulo y las de tus tablas al avanzar

- **Cada módulo tiene sus carpetas** (`src/app/solicitudes/`, `src/components/requests/`, `src/services/requests.js`…). Trabajar en lo tuyo casi nunca genera conflictos.
- **Puntos de integración** (un componente de otro dentro de tu página) se acuerdan antes: quién pone qué y dónde. Las guías ya los listan en «Coordinación con el equipo».

7.5 Comunicación

Canal	Para qué	Tiempo de respuesta esperado
Discord #daily	Avance diario asíncrono: qué hice, qué haré, qué me bloquea (plantilla en la Parte 10)	Escribir antes de las 10 p. m. los días que trabajes
Discord #ayuda	Bloqueos, con contexto completo (ver 7.6)	Menos de 2 horas en horario de trabajo
Discord #prs	Pedir revisión; avisar cambios de SQL en dev	Revisión en menos de 24 h

Canal	Para qué	Tiempo de respuesta esperado
Discord #decisiones [SUPUESTO]	Proponer decisiones de producto o técnicas; el resultado se registra en la sección 11	Opinar en menos de 48 h
WhatsApp	Solo urgencias (producción caída, demo en riesgo)	Lo antes posible

- **Avances visibles:** el tablero de GitHub Projects es la fuente de verdad. Una tarea que no está en el tablero no existe.
- **Malas noticias temprano:** si sabes que no llegas a una fecha, lo dices en la Daily en cuanto lo sepas, no el último día.
- **Ausencias:** si una semana vas a tener menos horas (parciales, viaje), avísalo en la Planning o con 2 días de anticipación.

7.6 Regla de atasco

1. **45 minutos** intentando solo (leer el error, buscar, preguntarle a la IA con contexto).
2. Escribes en **#ayuda**: qué intentabas, archivo, mensaje de error completo, qué probaste y una captura.
3. Si en **2 horas** nadie responde, mencionas a Diego.
4. Un bloqueo de **más de un día** se dice en la Daily y el SM lo convierte en prioridad del equipo.

Tema del bloqueo	Pregúntale primero a	Después a
Git, ramas, conflictos	Daniel	Diego
Diseño, layout, Tailwind	Daniel	Diego
Base de datos y SQL	Wilson	Diego
Políticas RLS y permisos	Iker	Wilson
Autenticación, sesión, roles	Diego	Iker
Vercel, despliegue, variables	Diego	Daniel
Formularios y Server Actions	Wilson	Diego

7.7 Uso de la IA en el equipo

- **No se pega código sin entenderlo.** Regla de fusión: si el autor no puede explicar su PR en la revisión, no se fusiona.
- **Contexto primero:** al iniciar cada conversación, pegar el documento de contexto y decir el stack: JavaScript, Next.js App Router, Tailwind, Supabase con `@supabase/ssr`, Vercel.
- **Revisar lo generado:** descartar código con TypeScript, la carpeta `pages/`, Flask u otros frameworks, paquetes nuevos o claves en el frontend. Probar siempre los permisos: la IA suele olvidar RLS y validaciones en el servidor.
- **Pedir explicaciones, no solo código:** «explícame qué hace cada parte» y «qué podría salir mal».
- **Nunca pegar claves ni contraseñas** en la IA.
- **Mantener el documento de contexto al día** (sección 8 al terminar módulos, 9 y 10 al encontrar errores, 11 con cada decisión): si está desactualizado, la IA da respuestas equivocadas para todos.

7.8 Cómo se toman y registran las decisiones

Tipo de decisión	Quién decide	Cómo
Producto (qué se construye, prioridades, reglas de negocio)	Diego (PO)	Escucha al equipo y a las empresas; decide
Técnica que afecta a todos (esquema, convenciones, paquetes)	El equipo	Propuesta en #decisiones; si no hay acuerdo en 48 h, decide Diego (en esquema, Wilson propone)
Técnica dentro de tu módulo	El dueño del módulo	Si no afecta a otros; se menciona en el PR
Proceso (ceremonias, acuerdos)	El equipo	En la Retrospectiva

Dónde se registran: sección 11 del documento de contexto, con fecha, decisión, alternativas y motivo (plantilla en la Parte 10). Decisiones ya tomadas que hay que registrar esta semana:

ID	Decisión	Estado
D-01	Reparto: Diego M1 y M4 · Wilson M3 y M5 · Iker M7 y M6 · Daniel M0 y M2	Tomada
D-02	Entrega a finales de enero (vie 29 ene); MVP v1 el vie 18 dic; descanso del 19 dic al 3 ene	Tomada (confirmar fecha con el profesor)
D-03	Dos proyectos de Supabase (dev y prod); SQL en prod solo Wilson y Diego (ver D-10)	Tomada
D-04	RLS automático activado; toda vista la revisan Wilson e Iker	Tomada
D-05	Main protegida; fusión solo con merge commit (por Vercel Hobby)	Tomada
D-06	`category_id` como clave foránea; `is_hidden` en contenido moderable; cancelar u ocultar en vez de borrar	Propuesta [SUPUESTO]
D-07	Mensajes solo entre el cliente y los proveedores que cotizaron su solicitud	Propuesta [SUPUESTO]
D-08	Reseña solo del cliente que aceptó una cotización, una por solicitud	Propuesta (sección 16)
D-09	Correos de confirmación: desactivar «Confirm email» en el MVP o SMTP propio	**Pendiente**
D-10	Wilson responsable de la base de datos (transición con Diego en el Sprint 0 y el Sprint 1); Daniel coordina la documentación	Tomada (30 oct)